

Jagiellonian University
Department of Theoretical Computer Science

Mateusz Przebieracz

Algorithms on chordal graphs

Bachelor Thesis

Supervisor: dr hab. Krzysztof Turowski

July 2026

Contents

1	Introduction	3
1.1	Chordal graph information	3
1.2	Basic definitions	4
1.3	Chordal graph	5
1.4	Perfect elimination ordering and followers	5
1.5	Property P	6
2	Chordal graph recognition algorithm of Rose, Tarjan, and Lueker	8
2.1	Introduction	8
2.2	Idea	8
2.3	Function LexBFS	8
2.3.1	Idea	8
2.3.2	Implementation	9
2.3.3	Pseudocode	9
2.4	Function FILL	9
2.4.1	Idea	9
2.4.2	Implementation	10
2.4.3	Pseudocode	10
2.5	Correctness	11
2.6	Complexity	11
3	Chordal graph recognition algorithm of Tarjan and Yannakakis	13
3.1	Introduction	13
3.2	Idea	13
3.3	Function MCS	13
3.3.1	Idea	13
3.3.2	Implementation	14
3.3.3	Pseudocode	14
3.4	Function ZERO-FILL-IN-TEST	14
3.4.1	Idea	14
3.4.2	Implementation	15
3.4.3	Pseudocode	15
3.5	Correctness	16
3.6	Complexity	17

4	Some optimization problems on chordal graphs	18
4.1	Introduction	18
4.2	Maximum Clique	18
4.2.1	Idea	18
4.2.2	Implementation	19
4.2.3	Pseudocode	19
4.2.4	Correctness	20
4.3	Minimum Vertex Coloring	20
4.3.1	Idea	20
4.3.2	Implementation	20
4.3.3	Pseudocode	21
4.3.4	Correctness	21
4.4	Maximum Independent Set and Minimum Clique Cover	22
4.4.1	Idea	22
4.4.2	Implementation	22
4.4.3	Pseudocode	23
4.4.4	Correctness	23
4.5	Complexity	24
5	Implementation	25
5.1	Introduction	25
5.2	Implementation details	25
5.3	Testing	26
5.4	Running time	26
5.4.1	Chordal Recognition: MCS vs. LexBFS	27
5.4.2	Optimization Algorithms	27
5.4.3	The Impact of Density	29
5.4.4	Constant Factor Analysis	30
5.5	Conclusions	31
	Bibliography	31

Chapter 1

Introduction

1.1 Chordal graph information

Chordal graphs, also known as triangulated graphs or rigid circuit graphs, form one of the most widely studied classes of perfect graphs. Early foundational work on chordal graphs was conducted by Dirac (1961) [1], who first characterized them in terms of simplicial vertices, and by Fulkerson and Gross (1965) [2].

The subclasses of chordal graphs include trees, interval graphs, split graphs, and strongly chordal graphs. Regarding the superclasses of chordal graphs, they encompass perfect graphs, weakly chordal graphs, and hole-free graphs [3].

An important application of chordal graphs arises in numerical linear algebra, specifically in the study of Gaussian elimination on sparse symmetric matrices, where they model zero fill-in [4]. They also play a significant role in relational database theory for representing acyclic database schemes [5].

This thesis focuses on the implementation and analysis of linear-time algorithms for chordal graphs. We begin by examining two foundational recognition methods: Lexicographic Breadth-First Search (LexBFS) and Maximum Cardinality Search (MCS). Both algorithms confirm a graph's chordality by constructing a Perfect Elimination Ordering (PEO). Building on this, we show how the PEO can be used to solve several classical graph problems in linear time, specifically finding the maximum clique, minimum vertex coloring, maximum independent set, and minimum clique cover.

While the PEO allows for efficient solutions to many graph problems that are otherwise difficult on general graphs, other problems remain computationally demanding even for this class. Table 1.1 summarizes the computational complexity of various well-known graph problems when restricted to chordal graphs. The algorithms that we implement and discuss in this work are marked in **bold**.

Problem	Complexity
Recognition	Linear [4, 5]
Maximum Clique	Linear [6]
Minimum Vertex Coloring	Linear [6]
Maximum Independent Set	Linear [6]
Minimum Clique Cover	Linear [6]
Treewidth Decomposition	Polynomial [7]
Feedback Vertex Set	Polynomial [8]
Graph Isomorphism	GI-complete [9]
Cutwidth Decomposition	NP-complete [10]
Domination	NP-complete [11]
Hamiltonian Cycle / Path	NP-complete [12]
Maximum Cut	NP-complete [13]

Table 1.1: Computational complexity of selected graph problems for chordal graphs. The algorithms implemented in this work are highlighted in bold.

1.2 Basic definitions

The following standard graph theory definitions are sourced from Bollobás’ “Modern Graph Theory” [14]. We begin by defining the fundamental components of a graph and its substructures.

Definition 1 (Graph). *A graph G is an ordered pair of disjoint sets (V, E) such that V is a set of vertices and E is a subset of the set $\binom{V}{2}$ of unordered pairs of V . We typically denote the number of vertices by $n = |V|$ and the number of edges by $m = |E|$.*

Definition 2 (Subgraph). *A graph $G' = (V', E')$ is a subgraph of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E$.*

Definition 3 (Induced subgraph). *If $G' = (V', E')$ is a subgraph of G and G' contains all the edges of G that join two vertices in V' , then G' is said to be an induced subgraph of G or spanned by V' and it is denoted by $G[V']$.*

Next, we formalize the concepts of vertex connectivity and graph traversal.

Definition 4 (Neighbors). *We denote a set of neighbors of vertex v in a graph $G = (V, E)$ by $N(v) = \{x : \{v, x\} \in E\}$. The degree of a vertex v , denoted as $\deg(v)$, is the size of its neighborhood, i.e., $\deg(v) = |N(v)|$.*

Definition 5 (Path). *A path is a sequence of distinct vertices $v_1, v_2, \dots, v_k$ such that $\{v_i, v_{i+1}\} \in E$ for $1 \leq i < k$.*

Definition 6 (Cycle). *If a path $v_1, v_2, \dots, v_k$ is such that $k \geq 3$ and $\{v_k, v_1\} \in E$, then it is said to be a cycle. The length of a cycle is the number of its vertices.*

Having established the basic structural components of a graph, we now introduce the specific subsets and numerical parameters associated with four classical graph optimization problems.

Definition 7 (Clique and Clique Number). *A clique in a graph G is a subset of vertices $C \subseteq V$ such that every two distinct vertices in C are adjacent. The clique number of G , denoted by $\omega(G)$, is the number of vertices in a maximum clique of G .*

Definition 8 (Independent Set and Independence Number). *An independent set in a graph G is a subset of vertices $I \subseteq V$ such that no two vertices in I are adjacent. The independence number of G , denoted by $\alpha(G)$, is the number of vertices in a maximum independent set of G .*

Definition 9 (Vertex Coloring and Chromatic Number). *A proper vertex coloring of a graph G is an assignment of colors to the vertices of V such that no two adjacent vertices share the same color. The chromatic number of G , denoted by $\chi(G)$, is the minimum number of colors required for a proper vertex coloring of G .*

Definition 10 (Clique Cover and Clique Cover Number). *A clique cover of a graph G is a partition of the vertex set V into subsets, such that each subset induces a clique in G . The clique cover number of G , denoted by $k(G)$, is the minimum number of cliques needed to cover all vertices of G .*

1.3 Chordal graph

We continue by defining the class of chordal graphs.

Definition 11 (Chord). *Let $C = v_1, v_2, \dots, v_k$ be a cycle in a graph G . A chord of C is an edge $\{v_i, v_j\} \in E$ that connects two non-consecutive vertices of the cycle.*

Definition 12 (Chordal graph). *A graph G is chordal if every cycle of length at least 4 has a chord.*

Equivalently, a graph is chordal if and only if it does not contain any induced cycle of length 4 or greater (i.e., it is hole-free).

1.4 Perfect elimination ordering and followers

The algorithmic recognition of chordal graphs relies heavily on the concept of vertex orderings. Let $\alpha: V \rightarrow \{1, 2, \dots, n\}$ be a bijection defining a total ordering of the vertices of G . We will use the notation $v <_\alpha w$ to denote that $\alpha(v) < \alpha(w)$.

Relative to an ordering α , the neighborhood $N(v)$ of a vertex v can be partitioned into two disjoint sets:

- $N^+(v) = \{w \in N(v) : v <_\alpha w\}$, which denotes the set of neighbors of v that appear after v in the ordering,
- $N^-(v) = \{w \in N(v) : w <_\alpha v\}$, which denotes the set of neighbors of v that appear before v in the ordering.

Definition 13 (Perfect elimination ordering [4]). *A perfect elimination ordering (PEO) of a graph $G = (V, E)$ is an ordering of its vertices such that for every vertex v , $N^+(v)$ induces a clique in G .*

The cornerstone of chordal graph recognition algorithms is the following theorem, originally proven by Dirac:

Theorem 1.4.1. [1] *A graph G is a chordal graph if and only if it has a perfect elimination ordering.*

When verifying if a generated ordering is a PEO, checking every pair of vertices in $N^+(v)$ would be computationally expensive. Instead, both algorithms for recognition rely on a more efficient structural property using the concept of a *follower*.

Definition 14 (Follower). *Let α be an ordering of vertices in G . For any vertex v where $N^+(v) \neq \emptyset$, its follower $f(v)$ is the vertex in $N^+(v)$ with the minimum α value, i.e., $f(v) = \arg \min_u \{\alpha(u) : u \in N^+(v)\}$.*

Lemma 1.4.2 (PEO-Follower Lemma). *An ordering α is a perfect elimination ordering of $G = (V, E)$ if and only if for every vertex $v \in V$ with $N^+(v) \neq \emptyset$, and for every $w \in N^+(v) \setminus \{f(v)\}$, the edge $\{f(v), w\} \in E$.*

Proof. ($\implies$) If α is a PEO, then $N^+(v)$ induces a clique. Since $f(v)$ and w are distinct vertices in $N^+(v)$, they must be adjacent.

($\impliedby$) We prove by induction on $i = \alpha(v)$ from n down to 1 that $N^+(v)$ is a clique. The base cases $i = n$ and $i = n - 1$ are trivial, as $|N^+(v)| \leq 1$. Assume the claim holds for all vertices with $\alpha(x) > i$, meaning that the ordering restricted to the subgraph induced by vertices $\{\alpha^{-1}(n), \dots, \alpha^{-1}(i+1)\}$ is a valid PEO. Consider the vertex v with $\alpha(v) = i$.

Let w_1 and w_2 be distinct vertices in $N^+(v)$. By definition, $f(v)$ is the vertex in $N^+(v)$ minimizing α . If $f(v)$ is either w_1 or w_2 , then the lemma's condition directly ensures $\{w_1, w_2\} \in E$. Otherwise, the condition implies $\{f(v), w_1\} \in E$ and $\{f(v), w_2\} \in E$, making both w_1 and w_2 higher-numbered neighbors of $f(v)$. Since $\alpha(f(v)) > \alpha(v) = i$, the inductive hypothesis ensures that $N^+(f(v))$ induces a clique. Consequently, w_1 and w_2 must be adjacent. Thus, $N^+(v)$ is a clique. The entire ordering α is confirmed to be a PEO once the induction reaches 1. $\square$

1.5 Property P

To generate a PEO, popular greedy algorithms traverse the graph in a specific way. Tarjan and Yannakakis [5] generalized the correctness of these traversals by defining a structural invariant we will refer to as *Property P*.

Definition 15 (Property P). *An ordering α satisfies Property P if, for any three vertices $u <_\alpha v <_\alpha w$ where $\{u, w\} \in E$ and $\{v, w\} \notin E$, there exists a vertex x such that $v <_\alpha x$, $\{v, x\} \in E$, and $\{u, x\} \notin E$.*

Theorem 1.5.1. *If a graph G is chordal, any ordering α satisfying Property P is a perfect elimination ordering.*

Proof. Assume G is chordal and α has Property P. We prove by contradiction that α is a PEO.

If α is not a PEO, then there must exist a vertex whose higher-numbered neighbors do not induce a clique. Thus, there exist vertices u, v , and w such that $u <_\alpha v <_\alpha w$, with $\{u, v\} \in E$ and $\{u, w\} \in E$, but $\{v, w\} \notin E$. This forms a chordless path w, u, v of length 2. If we denote $v_0 = w$, $v_1 = u$, and $v_2 = v$, we observe that $v_0 >_\alpha v_2 >_\alpha v_1$ and $v_1 <_\alpha v_2$.

Let us generalize this structure. We define a *V-shaped chordless path* as a chordless path $P = v_0, v_1, \dots, v_k$ ($k \geq 2$) such that for some index $i \in \{1, 2, \dots, k-1\}$, the elimination numbers strictly decrease from the endpoints down to v_i :

$$v_0 >_\alpha v_k >_\alpha v_1 >_\alpha v_2 > \dots >_\alpha v_i \quad \text{and} \quad v_i <_\alpha v_{i+1} < \dots <_\alpha v_k$$

Since the path w, u, v (with $k = 2$ and $i = 1$) satisfies these inequalities, the set of V-shaped chordless paths is non-empty. Among all such paths, choose one that maximizes the elimination number of its second-highest endpoint, $\alpha(v_k)$.

Since v_0 and v_1 are adjacent but v_0 and v_k are not, we have the triplet $v_1 <_\alpha v_k <_\alpha v_0$ with $\{v_1, v_0\} \in E$ and $\{v_k, v_0\} \notin E$. By Property P, there exists a vertex x such that $v_k <_\alpha x$, $\{v_k, x\} \in E$, and $\{v_1, x\} \notin E$.

Let $j \geq 2$ be the minimum index such that v_j is adjacent to x (we know $j \neq 1$ because $\{v_1, x\} \notin E$). Because G is chordal, x cannot be adjacent to v_0 ; otherwise, the sequence $v_0, v_1, \dots, v_j, x, v_0$ would form a chordless cycle of length 4 or more. Thus, $\{v_0, x\} \notin E$.

Because x is adjacent to v_j but not to $v_0, \dots, v_{j-1}$, a new chordless path is formed between v_0 and x .

- If $v_0 >_\alpha x$, the path $v_0, v_1, \dots, v_j, x$ is a V-shaped chordless path.
- If $x >_\alpha v_0$, the path $x, v_j, v_{j-1}, \dots, v_0$ is a V-shaped chordless path.

In either case, the new path has a second-highest endpoint strictly greater than v_k (since both $x >_\alpha v_k$ and $v_0 >_\alpha v_k$). This contradicts our choice of P as the path maximizing $\alpha(v_k)$. Therefore, no such V-shaped chordless path can exist, which in turn means our initial assumption was false. Thus, α must be a perfect elimination ordering. $\square$

Chapter 2

Chordal graph recognition algorithm of Rose, Tarjan, and Lueker

2.1 Introduction

Let us consider the linear recognition algorithm for chordal graphs by Rose, Tarjan, and Lueker [4]. Historically, this was the first linear-time algorithm for chordal graph recognition. It relies on a specialized graph traversal technique known as Lexicographic Breadth-First Search (LexBFS).

2.2 Idea

The fundamental idea is that a graph is chordal if and only if it has a perfect elimination ordering (PEO). The algorithm generates a specific vertex ordering $\alpha: V \rightarrow \{1, \dots, n\}$ that is guaranteed to be a PEO if the graph is chordal. Once this ordering is generated, a second phase verifies whether the ordering actually produces zero fill-in. If it does, the graph is chordal; otherwise, it is not.

The algorithm consists of two parts: **LexBFS** (to generate the ordering) and **FILL** (to verify whether the ordering is a PEO).

2.3 Function LexBFS

2.3.1 Idea

LexBFS assigns an elimination ordering α from n down to 1. Each unnumbered vertex v maintains a label consisting of the elimination numbers of its already-numbered neighbors, sorted in decreasing order. At each step, the algorithm selects the unnumbered vertex with the lexicographically largest label.

2.3.2 Implementation

Explicitly maintaining and comparing lists of numbers would be too slow. To achieve an $O(n + m)$ time complexity, we group unnumbered vertices that have the exact same label into sets. These sets are kept in a queue, ordered lexicographically from highest to lowest label.

Initially, all vertices have an empty label and are placed in a single set. When a vertex v is numbered, we examine all its unnumbered neighbors. For each unnumbered neighbour w , we remove it from its current set S . If this is the first time we are removing a vertex from S during the processing of v , we create a new set S' immediately preceding S in the queue, and place w into S' . This guarantees that vertices gaining the new highest number in their label are moved ahead of those that do not, maintaining the lexicographic order.

2.3.3 Pseudocode

Algorithm 1 Lexicographic Breadth-First Search (LexBFS)

```

1: function LEXBFS( $G = (V, E)$ )
2:   Create a single set  $S_0$  containing all vertices  $V$ 
3:   Initialize a queue  $Q$  containing only  $S_0$ 
4:   for  $i \leftarrow n$  downto 1 do
5:     Let  $S$  be the first non-empty set in  $Q$ 
6:     Remove a vertex  $v$  from  $S$ 
7:      $\alpha[v] \leftarrow i$ ,  $\alpha^{-1}[i] \leftarrow v$ 
8:     for each unnumbered neighbor  $w$  of  $v$  do
9:       Let  $S_w$  be the set containing  $w$ 
10:      if a new set  $S'_w$  was not created for  $S_w$  during step  $i$  then
11:        Create a new set  $S'_w$  and insert it in  $Q$  just before  $S_w$ 
12:      end if
13:      Move  $w$  from  $S_w$  to  $S'_w$ 
14:    end for
15:    Remove any empty sets from  $Q$ 
16:  end for
17:  return  $\alpha, \alpha^{-1}$ 
18: end function

```

2.4 Function FILL

2.4.1 Idea

FILL tests for zero fill-in by simulating the elimination process. For each vertex v , its higher-numbered neighbors must form a clique. By Lemma 1.4.2, this implies that all such neighbors must also be neighbors of $f(v)$ – the follower of v . We propagate these edges to $f(v)$ and count them. If the total number of

edges in this simulated elimination graph exceeds the original number of edges, fill-in was created, meaning G is not chordal.

2.4.2 Implementation

We initialize lists $A[v]$ with the original higher-numbered neighbors and record the total initial edge count m_{orig} . For each v in elimination order, we use a boolean array **test** to simultaneously deduplicate $A[v]$, find $f(v)$, and count the unique edges. The unique neighbors (excluding $f(v)$) are then appended to $A[f(v)]$. If the accumulated edge count m_{elim} exceeds the original m_{orig} , α is not a PEO.

2.4.3 Pseudocode

Algorithm 2 Zero Fill-In Test (FILL)

```

1: function FILL( $G = (V, E), \alpha, \alpha^{-1}$ )
2:   Initialize lists  $A[v] = \{w \in N(v) : \alpha[v] < \alpha[w]\}$  for all  $v$ 
3:    $m_{\text{orig}} \leftarrow \sum |A[v]|$ 
4:   test  $\leftarrow$  boolean array of size  $n + 1$ , initialized to false
5:    $m_{\text{elim}} \leftarrow 0$ 
6:   for  $i \leftarrow 1$  to  $n - 1$  do
7:      $v \leftarrow \alpha^{-1}[i]$ 
8:      $k \leftarrow n$ , unique_A  $\leftarrow$  empty list
9:     for each  $w \in A[v]$  do
10:      if not test $[\alpha[w]]$  then
11:        test $[\alpha[w]] \leftarrow$  true
12:        unique_A.append( $w$ )
13:         $k \leftarrow \min\{k, \alpha[w]\}$ 
14:      end if
15:    end for
16:     $m_{\text{elim}} \leftarrow m_{\text{elim}} + \text{unique\_A.size}()$ 
17:    if  $m_{\text{elim}} > m_{\text{orig}}$  then return NOT_CHORDAL
18:    end if
19:     $f(v) \leftarrow \alpha^{-1}[k]$ 
20:    for each  $w \in \text{unique\_A}$  do
21:      test $[\alpha[w]] \leftarrow$  false
22:      if  $w \neq f(v)$  then  $A[f(v)].\text{append}(w)$ 
23:    end if
24:  end for
25:  end for
26:  return CHORDAL
27: end function

```

2.5 Correctness

To prove the correctness of the overall algorithm, we first show that **LexBFS** generates a perfect elimination ordering, and then we show that **FILL** correctly verifies it.

Lemma 2.5.1. *Any ordering α generated by **LexBFS** satisfies Property P.*

Proof. Suppose $u <_\alpha v <_\alpha w$ with $\{u, w\} \in E$ and $\{v, w\} \notin E$. When **LexBFS** selects v to be numbered, w has already been numbered (since $v <_\alpha w$), and u is unnumbered. Let $L(y)$ denote the lexicographic label of an unnumbered vertex y . Because $\{u, w\} \in E$ and $\{v, w\} \notin E$, the elimination number $\alpha(w)$ is present in $L(u)$ but not in $L(v)$.

However, the algorithm chooses to number v before u , meaning $L(v)$ is lexicographically greater than or equal to $L(u)$. For this to be true despite $\alpha(w) \in L(u)$, there must be some elimination number strictly greater than $\alpha(w)$ that is present in $L(v)$ but not in $L(u)$. Let this number belong to a vertex x . Since $\alpha(x) > \alpha(w) > \alpha(v)$, x was numbered before v . Thus, $v <_\alpha x$, $\{v, x\} \in E$, and $\{u, x\} \notin E$, satisfying Property P. $\square$

Theorem 2.5.2. *If G is a chordal graph, the ordering α generated by **LexBFS** is a perfect elimination ordering.*

Proof. Follows immediately from the previous lemma and Theorem 1.5.1. $\square$

Theorem 2.5.3. *The **FILL** algorithm correctly determines whether α is a perfect elimination ordering.*

Proof. By Lemma 1.4.2, α is a PEO if and only if for every v , all of its higher-numbered neighbors w are connected to the follower $f(v)$.

The **FILL** algorithm mathematically simulates the creation of an elimination graph G^* by enforcing this lemma. For each vertex v , the algorithm identifies the follower $f(v)$ (denoted in the code as $\alpha^{-1}[k]$) and adds the required edges $\{w, f(v)\}$ to the adjacency lists. It accumulates the total number of unique edges required to satisfy the lemma into m_{elim} .

If α is a PEO, no new edges are required; every propagated edge already exists in the graph, and m_{elim} will exactly equal the original number of higher-numbered edges, m_{orig} . If $m_{\text{elim}} > m_{\text{orig}}$, it implies that a missing edge was propagated to enforce the clique condition, meaning fill-in was generated, and G is not chordal. $\square$

2.6 Complexity

Theorem 2.6.1. *The total time complexity of the algorithm is $O(n + m)$.*

Proof. Let us analyze the complexity of the two phases separately:

- **LexBFS**: By implementing the queue of sets and the sets themselves as doubly-linked lists, all set operations take constant time. Each vertex is added to and removed from a set exactly once, taking $O(1)$ time. The inner loop processes each edge $\{v, w\}$ exactly twice. Moving a vertex to a newly created preceding set involves basic pointer operations in a doubly linked list, which is $O(1)$. Thus, the **LexBFS** phase runs in $O(n + m)$ time.
- **FILL**: We simulate the elimination process by maintaining adjacency lists $A[v]$ of higher-numbered neighbors. The use of the **test** array allows us to eliminate duplicates in time proportional to $|A[v]|$. Because the algorithm aborts and returns **NOT_CHORDAL** the moment the total number of edges m_{elim} in the elimination graph exceeds the original number of edges m_{orig} , the maximum number of elements appended to, and subsequently processed from, all $A[v]$ lists across the entire execution is strictly bounded by m_{orig} . Since maintaining the **test** array and updating elements takes $O(1)$ time per element, this phase also runs in strictly $O(n + m)$ time.

Combining both parts, the overall time complexity of the chordal graph recognition algorithm is $O(n + m)$, and the space complexity is $O(n + m)$ to store the graph, the sets, and the simulated adjacency lists. $\square$

Chapter 3

Chordal graph recognition algorithm of Tarjan and Yannakakis

3.1 Introduction

Let us consider a linear recognition algorithm for chordal graphs by Tarjan and Yannakakis [5]. While LexBFS solved the recognition problem in linear time, Tarjan and Yannakakis later introduced a simplified approach based on a new search strategy called Maximum Cardinality Search (MCS). Its main ideas are highly intuitive and avoid the complex queue-of-sets data structure required for lexicographic sorting, replacing it with straightforward bucket-based arrays.

3.2 Idea

Similar to LexBFS, the algorithm depends on finding a perfect elimination ordering. The algorithm consists of two distinct parts: MCS computes an ordering α that is guaranteed to be a PEO if G is chordal. Then, **ZERO-FILL-IN-TEST** verifies whether this ordering actually produces zero fill-in. Let us look at each of the parts in turn.

3.3 Function MCS

3.3.1 Idea

Maximum Cardinality Search numbers the vertices from n down to 1. Instead of maintaining sorted labels of numbered neighbors, it simply counts them. As the next vertex to number, we select an unnumbered vertex adjacent to the largest number of previously numbered vertices.

3.3.2 Implementation

To implement the MCS efficiently in linear time, we maintain an array of sets $\text{set}[i]$ containing all unnumbered vertices adjacent to exactly i numbered vertices. We track the maximum index j such that $\text{set}[j]$ is nonempty and keep track of the set a vertex is in with an array $\text{size}[v]$. When a vertex v is removed from $\text{set}[j]$ and numbered, we iterate over its unnumbered neighbors. A neighbor w currently in $\text{set}[k]$ is moved to $\text{set}[k + 1]$.

3.3.3 Pseudocode

Algorithm 3 Maximum Cardinality Search (MCS)

```

1: function MCS( $G = (V, E)$ )
2:   Initialize  $\text{set}[0 \dots n - 1]$  as empty doubly linked lists
3:   for each  $v \in V$  do
4:      $\text{size}[v] \leftarrow 0$ ,  $\text{set}[0].\text{add}(v)$ 
5:   end for
6:    $j \leftarrow 0$ 
7:   for  $i \leftarrow n$  downto 1 do
8:      $v \leftarrow \text{set}[j].\text{pop}()$ 
9:      $\alpha[v] \leftarrow i$ ,  $\alpha^{-1}[i] \leftarrow v$ 
10:     $\text{size}[v] \leftarrow -1$  ▷ Mark as processed
11:    for each neighbor  $w$  of  $v$  do
12:      if  $\text{size}[w] \geq 0$  then ▷ If  $w$  is unnumbered
13:         $\text{set}[\text{size}[w]].\text{remove}(w)$ 
14:         $\text{size}[w] \leftarrow \text{size}[w] + 1$ 
15:         $\text{set}[\text{size}[w]].\text{add}(w)$ 
16:      end if
17:    end for
18:     $j \leftarrow j + 1$ 
19:    while  $j \geq 0$  and  $\text{set}[j] = \emptyset$  do
20:       $j \leftarrow j - 1$ 
21:    end while
22:  end for
23:  return  $\alpha, \alpha^{-1}$ 
24: end function

```

3.4 Function ZERO-FILL-IN-TEST

3.4.1 Idea

Like the LexBFS FILL phase, this test verifies the PEO-Follower Lemma (Lemma 1.4.2): for every vertex v , its follower $f(v)$ must be connected to all other higher-numbered neighbors w .

However, Tarjan and Yannakakis achieve this much more elegantly. Instead of actively propagating adjacency lists forward and counting simulated edges, their algorithm processes each vertex w and looks backward at its lower-numbered neighbors v . By using a simple index array to mark the neighbors of w , it verifies the existence of the required $\{f(v), w\}$ edges on the fly in $O(1)$ time. This completely avoids dynamic allocations and edge counting, resulting in a significantly simpler and cleaner implementation.

3.4.2 Implementation

We process vertices $w = \alpha^{-1}[i]$ from $i = 1$ to n . We maintain two arrays: $f[v]$ initialized to v , and $\text{index}[v]$. For each lower-numbered neighbour v of w , we set $\text{index}[v] = i$ and, if v does not yet have a follower, we assign $f[v] = w$.

Then, we loop through those same lower-numbered neighbors v again. For G to be chordal, the designated follower $f[v]$ must be connected to w . We verify this by checking if $\text{index}[f[v]] = i$. If it is not, a fill-in edge is required, and the graph is not chordal.

3.4.3 Pseudocode

Algorithm 4 Zero Fill-In Test

```

1: function TEST-ZERO-FILL-IN( $G = (V, E), \alpha, \alpha^{-1}$ )
2:   for  $v \in V$  do
3:     Initialize  $f[v] \leftarrow v$  and  $\text{index}[v] \leftarrow 0$ 
4:   end for
5:   for  $i \leftarrow 1$  to  $n$  do
6:      $w \leftarrow \alpha^{-1}[i]$ 
7:      $\text{index}[w] \leftarrow i$ 
8:     for each neighbor  $v$  of  $w$  with  $\alpha[v] < i$  do
9:        $\text{index}[v] \leftarrow i$ 
10:      if  $f[v] = v$  then  $f[v] \leftarrow w$ 
11:      end if
12:    end for
13:    for each neighbor  $v$  of  $w$  with  $\alpha[v] < i$  do
14:      if  $\text{index}[f[v]] < i$  then
15:        return NOT_CHORDAL ▷ Fill-in detected
16:      end if
17:    end for
18:  end for
19:  return CHORDAL
20: end function

```

3.5 Correctness

Similar to LexBFS, we prove the correctness of MCS by showing that it satisfies Property P, and that the ZERO-FILL-IN-TEST correctly evaluates the Follower condition from the Introduction.

Lemma 3.5.1. *Any ordering α generated by Maximum Cardinality Search satisfies Property P.*

Proof. Suppose $u <_\alpha v <_\alpha w$ with $\{u, w\} \in E$ and $\{v, w\} \notin E$. At the step when the algorithm selects v to be numbered, w is already numbered, while u remains unnumbered.

Because the algorithm chooses to number v before u , v must be adjacent to at least as many already-numbered vertices as u . We know that u is adjacent to the already-numbered vertex w , but v is not. Therefore, to make up for this difference in count, v must be adjacent to some other already-numbered vertex x that is not adjacent to u .

Since x is already numbered when v is selected, we have $v <_\alpha x$. Thus, we have established that $\{v, x\} \in E$ and $\{u, x\} \notin E$, which satisfies the definition of Property P. $\square$

Theorem 3.5.2. *If G is a chordal graph, the ordering α generated by MCS is a perfect elimination ordering.*

Proof. Follows immediately from the previous lemma and Theorem 1.5.1. $\square$

Theorem 3.5.3. *The ZERO-FILL-IN-TEST algorithm correctly determines whether α is a perfect elimination ordering.*

Proof. By Lemma 1.4.2, α is a PEO if and only if for every vertex v and every higher-numbered neighbor $w \neq f(v)$, the edge $\{f(v), w\}$ exists. We can view this requirement from the perspective of w : for each vertex w and each of its lower-numbered neighbors v , if $w \neq f(v)$, then w must be adjacent to $f(v)$.

The ZERO-FILL-IN-TEST directly validates this. The algorithm processes each vertex $w = \alpha^{-1}[i]$ in increasing elimination order.

In the first inner loop, it iterates over all lower-numbered neighbors v of w , assigning $\text{index}[v] \leftarrow i$. This effectively turns the index array into an $O(1)$ lookup table for the neighborhood of w : for any vertex $x <_\alpha w$, $\text{index}[x] = i$ if and only if $\{x, w\} \in E$. During this loop, the algorithm also correctly identifies and assigns the follower $f(v) = w$ the first time a higher-numbered neighbor of v is encountered.

In the second inner loop, it iterates over the same lower-numbered neighbors v of w and evaluates $\text{index}[f[v]]$. Because $f(v) \leq_\alpha w$, checking if $\text{index}[f[v]] = i$ is equivalent to checking whether the required edge $\{f(v), w\}$ exists in G (or if $f(v) = w$). If the value is strictly less than i , the required edge is missing, violating Lemma 1.4.2. The algorithm immediately detects this and correctly concludes that α produces fill-in and G is not chordal. $\square$

3.6 Complexity

Theorem 3.6.1. *The total time complexity of the algorithm is $O(n + m)$.*

Proof. Let us analyze the complexity of the two phases separately:

- **MCS:** The initialization of the structures takes $O(n)$ time. The outer loop executes n times. The inner loop iterates over the neighbors of the current vertex v . Moving a vertex w from one set to another takes $O(1)$ time due to the doubly linked list implementation. The variable j is incremented at most m times (once per edge) and decremented at most $m + n$ times overall since it is bounded by n and never drops below 0. Therefore, the total time for MCS is $O(\sum_{v \in V} \deg(v) + n) = O(n + m)$.
- **ZERO-FILL-IN-TEST:** The outer loop runs n times. The inner loops iterate over the neighbors of w that occur before w in the ordering. Each edge $\{v, w\}$ is processed exactly twice (once in each inner loop) when the higher-numbered endpoint w is visited. All operations inside the loops, including checking and updating the arrays f and **index**, are performed in $O(1)$ time. Thus, the total time complexity of this function is $O(n + m)$.

Combining both parts, the overall time complexity of the chordal graph recognition algorithm is $O(n + m)$, and the space complexity is $O(n + m)$ to store the graph and the sets. $\square$

Chapter 4

Some optimization problems on chordal graphs

4.1 Introduction

In the general study of algorithmic graph theory, classical optimization problems such as finding the maximum clique, minimum vertex coloring, maximum independent set, and minimum clique cover are known to be NP-complete. However, chordal graphs belong to the family of *perfect graphs*, which inherently satisfy the strong duality relationships $\omega(G) = \chi(G)$ and $\alpha(G) = k(G)$. [15]

More importantly, from a computational perspective, the existence of a Perfect Elimination Ordering (PEO) allows us to bypass the NP-completeness of these problems entirely. The foundational work translating the mathematical properties of chordal graphs into linear-time optimization algorithms was introduced by Fănică Gavril in 1972 [6].

In this chapter, we assume that a recognition algorithm (such as LexBFS or MCS) has already been executed on the chordal graph $G = (V, E)$, providing a valid perfect elimination ordering α alongside its inverse array α^{-1} . By utilizing $N^+(v)$, we will demonstrate how to solve all four of these classical problems in a single pass over the ordering.

4.2 Maximum Clique

4.2.1 Idea

In a general graph, finding a maximum clique requires an exhaustive search. In a chordal graph, the PEO completely localizes this search. Because every vertex v together with $N^+(v)$ forms a clique by the definition of a PEO, every maximal clique in the graph is uniquely anchored by the vertex within it that appears earliest in the elimination ordering. Consequently, to find the maximum clique,

we simply need to evaluate the size of the clique $\{v\} \cup N^+(v)$ for all $v \in V$ and select the largest one.

4.2.2 Implementation

Assuming we have the PEO α , determining the size of the maximum clique is straightforward. We initialize a variable $\omega_{max} = 0$. We then iterate over every vertex $v \in V$, isolating its higher-numbered neighbors to compute $|N^+(v)|$. The size of the clique initiated by v is exactly $|N^+(v)| + 1$. We update ω_{max} if this value is strictly greater than the current ω_{max} . To extract the actual vertices of the maximum clique (rather than just its size), we simply remember the vertex v_{max} that yielded the maximum value. The maximum clique itself is exactly $\{v_{max}\} \cup N^+(v_{max})$.

4.2.3 Pseudocode

Algorithm 5 Find Maximum Clique

```

1: function MAXIMUM-CLIQUE( $G = (V, E), \alpha$ )
2:    $\omega_{max} \leftarrow 0$ 
3:    $v_{max} \leftarrow \text{null}$ 
4:   for each  $v \in V$  do
5:      $N^+ \leftarrow \text{empty list}$ 
6:     for each neighbor  $w$  of  $v$  do
7:       if  $\alpha[v] < \alpha[w]$  then
8:          $N^+.append(w)$ 
9:       end if
10:    end for
11:     $current\_size \leftarrow N^+.size() + 1$ 
12:    if  $current\_size > \omega_{max}$  then
13:       $\omega_{max} \leftarrow current\_size$ 
14:       $v_{max} \leftarrow v$ 
15:    end if
16:  end for
17:   $MaxClique \leftarrow \text{empty list}$ 
18:   $MaxClique.append(v_{max})$ 
19:  for each neighbor  $w$  of  $v_{max}$  do
20:    if  $\alpha[v_{max}] < \alpha[w]$  then
21:       $MaxClique.append(w)$ 
22:    end if
23:  end for
24:  return  $MaxClique, \omega_{max}$ 
25: end function

```

4.2.4 Correctness

Theorem 4.2.1. *The MAXIMUM-CLIQUE algorithm correctly identifies the maximum clique of a chordal graph.*

Proof. Let C be an arbitrary maximal clique in the chordal graph G . Since C is a finite subset of vertices, there must exist a unique vertex $v \in C$ such that $\alpha(v) = \min_{w \in C} \alpha(w)$. Because v is the vertex in C with the smallest elimination number, every other vertex $w \in C$ satisfies $\alpha(v) < \alpha(w)$. Furthermore, since C is a clique, v is adjacent to all other vertices in C . Thus, $(C \setminus \{v\}) \subseteq N^+(v)$, which implies $C \subseteq \{v\} \cup N^+(v)$.

By the definition of a perfect elimination ordering, $\{v\} \cup N^+(v)$ is a clique in G . Since C is a maximal clique and $C \subseteq \{v\} \cup N^+(v)$, it must be that $C = \{v\} \cup N^+(v)$.

Because every maximal clique takes this exact form, the set of all maximal cliques in G is a subset of the collection of sets $\{\{v\} \cup N^+(v)\}_{v \in V}$. Therefore, calculating $|N^+(v)| + 1$ for all vertices and taking the maximum is guaranteed to find the size of the maximum clique of G . $\square$

4.3 Minimum Vertex Coloring

4.3.1 Idea

The standard First-Fit (or greedy) coloring algorithm sequentially assigns each vertex the smallest available color not already used by its colored neighbors. While First-Fit performs poorly on general graphs for arbitrary vertex orderings, it behaves perfectly on chordal graphs if the vertices are processed in the exact reverse order of the perfect elimination ordering (from n down to 1). In this specific sequence, the local constraints dynamically force a globally optimal coloring.

4.3.2 Implementation

To implement the First-Fit algorithm, we process the vertices $v = \alpha^{-1}[i]$ in decreasing order of i . When coloring v , the neighbors that have already been assigned a color are exactly the set $N^+(v)$.

We maintain an array `color[v]` initialized to 0. To find the smallest available color for v in $O(|N^+(v)|)$ time, we use an auxiliary boolean array `used`. We mark the colors of all vertices in $N^+(v)$ as **true** in the `used` array, scan sequentially for the first **false** index to assign to `color[v]`, and then reset the modified entries in the `used` array to prepare for the next iteration.

4.3.3 Pseudocode

Algorithm 6 Minimum Vertex Coloring

```

1: function FIRST-FIT( $G = (V, E), \alpha, \alpha^{-1}$ )
2:   color  $\leftarrow$  array of size  $n + 1$ , initialized to 0
3:   used  $\leftarrow$  boolean array of size  $n + 1$ , initialized to false
4:    $\chi_{max} \leftarrow 0$ 
5:   for  $i \leftarrow n$  downto 1 do
6:      $v \leftarrow \alpha^{-1}[i]$ 
7:     for each neighbor  $w$  of  $v$  do
8:       if  $\alpha[v] < \alpha[w]$  then
9:         used[color[ $w$ ]]  $\leftarrow$  true
10:      end if
11:    end for
12:    color[ $v$ ]  $\leftarrow \min\{c \in \mathbb{N}_+ : \text{used}[c] = \text{false}\}$ 
13:     $\chi_{max} \leftarrow \max\{\chi_{max}, \text{color}[v]\}$ 
14:    for each neighbor  $w$  of  $v$  do  $\triangleright$  Reset array for the next iteration
15:      if  $\alpha[v] < \alpha[w]$  then
16:        used[color[ $w$ ]]  $\leftarrow$  false
17:      end if
18:    end for
19:  end for
20:  return color,  $\chi_{max}$ 
21: end function

```

4.3.4 Correctness

Theorem 4.3.1. *The FIRST-FIT algorithm operating in reverse perfect elimination order produces an optimal coloring of a chordal graph, proving that $\chi(G) = \omega(G)$.*

Proof. It is a universal fact for all graphs that the chromatic number $\chi(G)$ must be at least the size of the maximum clique $\omega(G)$, because every vertex in a clique must receive a distinct color. Hence, $\chi(G) \geq \omega(G)$.

Consider the First-Fit algorithm running in reverse PEO. When the algorithm attempts to assign a color to a vertex v , the only constraints acting upon v come from the set of its already colored neighbors, which is exactly $N^+(v)$.

Because α is a perfect elimination ordering, the set $\{v\} \cup N^+(v)$ induces a clique in G . The size of this clique is $|N^+(v)| + 1$. By the definition of the maximum clique $\omega(G)$, no clique in G can exceed $\omega(G)$ in size. Therefore, $|N^+(v)| + 1 \leq \omega(G)$, which directly implies that $|N^+(v)| \leq \omega(G) - 1$.

Since v has at most $\omega(G) - 1$ already-colored neighbors, there will always be at least one available color in the range $\{1, 2, \dots, \omega(G)\}$.

Thus, the algorithm will successfully color the entire graph without ever requiring a color strictly greater than $\omega(G)$. This proves that $\chi(G) \leq \omega(G)$. Thus, the algorithm is optimal and $\chi(G) = \omega(G)$. $\square$

4.4 Maximum Independent Set and Minimum Clique Cover

4.4.1 Idea

Because an independent set contains vertices that are pairwise non-adjacent, no single clique can ever cover more than one vertex of an independent set. Thus, the size of any independent set is a natural lower bound for the size of any clique cover, establishing the universal inequality $\alpha(G) \leq k(G)$. If we can construct an independent set and a clique cover of the exact same size, we guarantee that both are perfectly optimal. Gavril [6] demonstrated that by greedily processing a chordal graph in the perfect elimination order (from 1 to n), both structures can be built simultaneously and optimally.

4.4.2 Implementation

We maintain a boolean array `covered` initialized to `false` for all vertices. We also initialize an empty list I to store the maximum independent set, and a list of lists K to store the minimum clique cover. We process the vertices $v = \alpha^{-1}[i]$ in increasing order of i from 1 to n .

If v is already marked as `covered`, we simply ignore it and continue. If v is not `covered`, it means that no clique generated so far has encompassed v . We immediately append v to our independent set I . At the same time, we construct a new clique $C_v = \{v\} \cup N^+(v)$ and append C_v to our clique cover K . Finally, we mark v and all of its neighbors in $N^+(v)$ as `covered`. Once the loop concludes, I contains the maximum independent set, and K contains the minimum clique cover.

4.4.3 Pseudocode

Algorithm 7 Max Independent Set & Min Clique Cover

```

1: function INDEPENDENT-SET-CLIQUE-COVER( $G = (V, E), \alpha, \alpha^{-1}$ )
2:   covered  $\leftarrow$  boolean array of size  $n + 1$ , initialized to false
3:    $I \leftarrow$  empty list  $\triangleright$  Maximum Independent Set
4:    $K \leftarrow$  empty list of lists  $\triangleright$  Minimum Clique Cover
5:   for  $i \leftarrow 1$  to  $n$  do
6:      $v \leftarrow \alpha^{-1}[i]$ 
7:     if not covered $[v]$  then
8:        $I.append(v)$ 
9:        $C_v \leftarrow$  empty list
10:       $C_v.append(v)$ 
11:      covered $[v] \leftarrow$  true
12:      for each neighbor  $w$  of  $v$  do
13:        if  $\alpha[v] < \alpha[w]$  then
14:           $C_v.append(w)$ 
15:          covered $[w] \leftarrow$  true
16:        end if
17:      end for
18:       $K.append(C_v)$ 
19:    end if
20:  end for
21:  return  $I, K$ 
22: end function

```

4.4.4 Correctness

Theorem 4.4.1. *The INDEPENDENT-SET-CLIQUE-COVER algorithm produces a valid independent set I and a valid clique cover K of equal size, proving that $\alpha(G) = k(G)$.*

Proof. We must verify three things: that K is a valid clique cover, that I is a valid independent set, and that their optimality is guaranteed.

1. K is a valid clique cover: Every subset C_v added to K is of the form $\{v\} \cup N^+(v)$. Because α is a PEO, this subset is a clique in G . Furthermore, every vertex in the graph is eventually marked as covered. A vertex is covered either because it triggered the creation of its own clique C_v , or because it was swept into the clique C_u of some earlier, lower-numbered neighbor u . Hence, the union of all cliques in K covers V .

2. I is a valid independent set: Suppose that two vertices $u, v \in I$ are adjacent. Because α is a total ordering, one must appear before the other. Without loss of generality, assume $\alpha(u) < \alpha(v)$. Since they are adjacent, v is a higher-numbered neighbor of u , meaning $v \in N^+(u)$.

When the algorithm processed u at step $\alpha(u)$, it was added to I , and the clique $C_u = \{u\} \cup N^+(u)$ was generated. During this step, every vertex in

C_u was permanently marked as **covered**. Since $v \in N^+(u)$, v was marked as covered at step $\alpha(u)$. Later, when the algorithm reached step $\alpha(v)$ to evaluate v , the condition **if not covered** $[v]$ would evaluate to **false**. Consequently, v could not possibly be added to I . This contradicts the assumption that both u and v are in I . Thus, no two vertices in I can be adjacent, and I is a valid independent set.

3. Optimality: By the mechanics of the algorithm, exactly one element is added to I each time a new clique is added to K . Therefore, $|I| = |K|$. As established earlier, the size of any independent set cannot exceed the size of any clique cover, meaning $|I| \leq \alpha(G) \leq k(G) \leq |K|$. Since $|I| = |K|$, this collapses into an equality: $|I| = \alpha(G) = k(G) = |K|$. This proves that both the generated independent set and the clique cover are optimal. $\square$

4.5 Complexity

Theorem 4.5.1. *The total time complexity for each of the optimization algorithms presented in this chapter is $O(n + m)$.*

Proof. This linear complexity stems from the fact that all algorithms utilize the exact same structural traversal mechanism. Processing the vertices in the order dictated by α (either forward or reverse) allows the algorithms to evaluate local constraints simply by inspecting the adjacency list $N(v)$ of each vertex v and filtering for higher-numbered neighbors $N^+(v)$.

Specifically:

- **Maximum Clique:** For each vertex v , identifying $N^+(v)$ requires a single iteration over the adjacency list of v . The size computation and maximum update take $O(1)$ time per vertex.
- **Minimum Vertex Coloring:** The algorithm iterates through $N(v)$ twice per vertex (once to mark the colors used by $N^+(v)$ and once to unmark them). Finding the lowest available color using the boolean array takes time proportional to the number of used colors, which is bounded by $\deg(v)$.
- **Independent Set and Clique Cover:** The algorithm iterates through $N(v)$ exactly once when an uncovered vertex v is encountered, in order to mark the vertices forming the newly covered clique as **covered**.

In all algorithms, the dominant operation is iterating over the adjacency lists. Over the course of the outer loop running n times, every adjacency list is traversed a constant number of times. The sum of the degrees of all vertices is $2m$. Combined with $O(n)$ array initializations, the total time complexity for any of these optimization passes is strictly bounded by $O(n + m)$. $\square$

Chapter 5

Implementation

5.1 Introduction

The algorithms described above were implemented in C++. For graph operations, the NetworkKit and Koala libraries were used. The code is open source and available in the Koala-NetworkKit GitHub repository at <https://github.com/krzysztof-turowski/koala-networkkit/pull/46>.

All tests and benchmarks were executed on a system equipped with an AMD Ryzen 7 7800X3D 8-Core Processor and 32GB of DDR5 RAM.

5.2 Implementation details

The repository contains separate classes for each type of problem, such as chordal graph recognition, maximum clique calculation, minimum vertex coloring, and so on. Below is a brief description of the files with their purposes:

- `koala-networkkit/include/recognition/ChordalGraphRecognition.hpp`
The main header file defining the classes for LexBFS and MCS chordal graph recognition algorithms.
- `koala-networkkit/include/coloring/ChordalVertexColoring.hpp`
Header file for the class that finds the graph coloring with the minimum number of colors.
- `koala-networkkit/include/clique/ChordalClique.hpp`
Header file for the class that finds the maximum clique of a graph.
- `koala-networkkit/include/clique_cover/ChordalCliqueCover.hpp`
Header file for the class that finds the minimum clique cover of a graph.
- `koala-networkkit/include/independent_set/ChordalIndependentSet.hpp`
Header file for the class that finds the maximum independent set of a graph.

- `koala-networkkit/benchmark/`
Folder containing benchmarking programs to evaluate the performance of the algorithms on various graph instances.
- `koala-networkkit/test/`
Folder containing unit tests for checking the correctness of the implemented algorithms.

5.3 Testing

The correctness of each algorithm was tested in two ways:

1. Exhaustive tests on all small-sized connected graphs, i.e., graphs with up to 10 vertices.
2. On graphs from a benchmark suite containing random chordal and non-chordal graphs with up to 10^7 vertices and edges.

To generate robust and structurally varied chordal graphs in linear time, we used the subtree-intersection generator proposed by Shalom et al. (2021) [16], available at <https://github.com/cmshalom/ChordalGraphGeneration>. Negative testing was performed using random Erdős-Rényi graphs to ensure the algorithms correctly reject non-chordal inputs.

Suite	Graph Topology	Graphs
T1: Exhaustive	All connected $n \leq 10$	11,989,764
T2: Medium Chordal	Shalom gen. ($n + m \sim 10^5$)	5,000
T3: Large Chordal	Shalom gen. ($n + m \sim 10^7$)	1,000
T4: Medium Random	Erdős-Rényi ($n = 1\text{K}, m = 5\text{K}$)	10,000
T5: Large Random	Erdős-Rényi ($n = 50\text{K}, m = 250\text{K}$)	1,000

Table 5.1: Summary of correctness test suites. T1 tested all possible graphs up to $n = 10$, identifying exactly 123,418 chordal graphs [17]. T4 and T5 verified that the recognition algorithms successfully reject non-chordal structures.

5.4 Running time

The algorithms were tested on several subgroups of tests. Each data point consists of the median runtime of 3 repetitions.

To demonstrate asymptotic scalability, the algorithms were tested across three density tiers: Sparse ($m \in O(n)$), Medium ($m \in O(n^{1.5})$), and Dense ($m \in O(n^2)$). The median runtimes, in milliseconds, are presented in Table 5.2 and Table 5.3.

5.4.1 Chordal Recognition: MCS vs. LexBFS

Table 5.2 and Figure 5.1 compare the two chordal graph recognition algorithms: Maximum Cardinality Search (MCS) and Lexicographic Breadth-First Search (LexBFS).

	Sparse		Medium		Dense	
$n + m$	MCS	LexBFS	MCS	LexBFS	MCS	LexBFS
10^4	0.238	0.682	0.079	0.231	0.036	0.093
10^5	1.96	5.18	0.573	1.38	0.805	2.39
10^6	14.3	36.5	7.79	20.2	6.08	21.6
10^7	672	1349	61.7	178	64.0	215

Table 5.2: The running time (in ms) of the chordal recognition algorithms across density tiers.

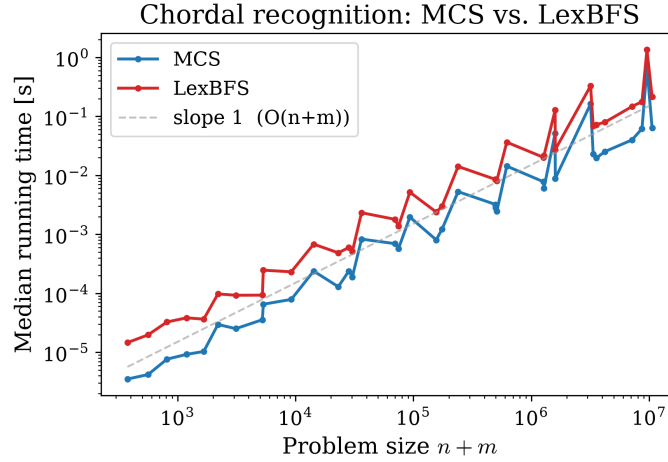


Figure 5.1: Graph of the running time of the recognition algorithms.

5.4.2 Optimization Algorithms

Table 5.3 shows the running times of the four PEO-based optimization algorithms. The time required to compute the PEO is excluded. The scaling behaviors across the sparse, medium, and dense density tiers are plotted sequentially in Figure 5.2, Figure 5.3, and Figure 5.4.

$n + m$	Coloring	Max Clique	Clique Cover	Ind. Set
Medium ($m \in O(n^{1.5})$)				
10^4	0.036	0.010	0.011	0.002
10^5	0.198	0.081	0.034	0.011
10^6	2.52	1.27	0.255	0.101
10^7	28.1	17.9	1.13	0.576
Dense ($m \in O(n^2)$)				
10^4	0.013	0.004	0.002	0.001
10^5	0.265	0.120	0.006	0.002
10^6	1.86	0.870	0.019	0.005
10^7	20.9	13.7	0.083	0.033

Table 5.3: The running time (in ms) of the optimization algorithms across the medium and dense density tiers.

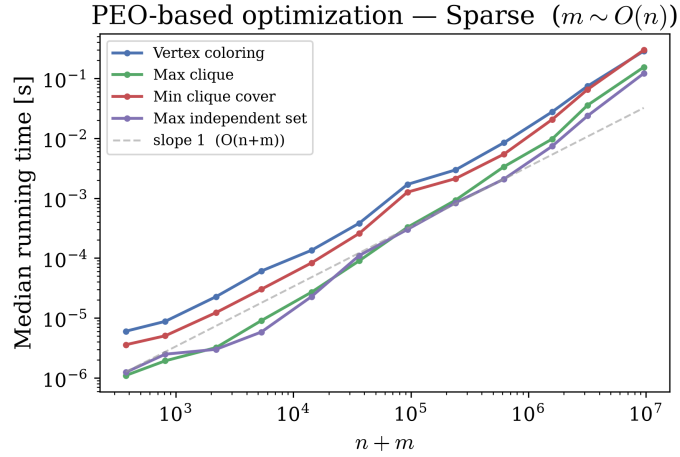


Figure 5.2: Graph of the running time of the optimization algorithms on the sparse density tier.

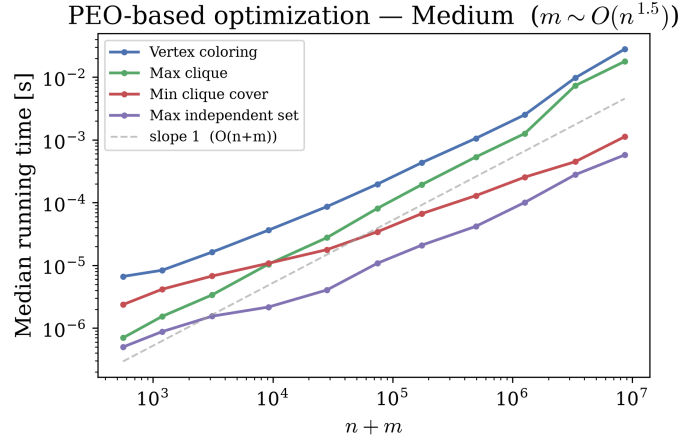


Figure 5.3: Graph of the running time of the optimization algorithms on the medium density tier.

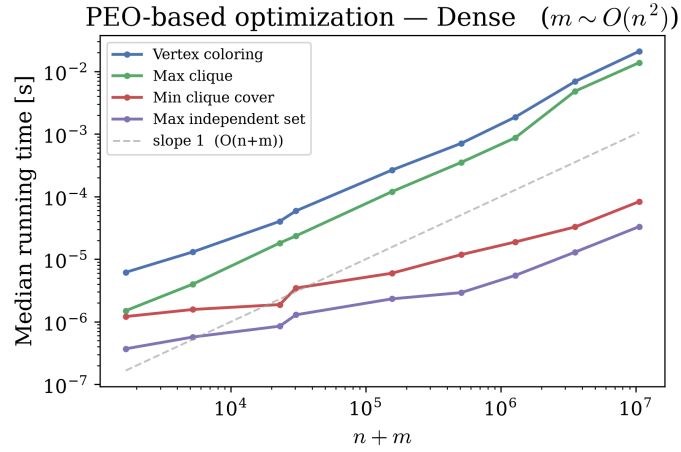


Figure 5.4: Graph of the running time of the optimization algorithms on the dense density tier.

5.4.3 The Impact of Density

To isolate the impact of the number of edges, an additional benchmark was performed with a fixed vertex count of $n = 5000$, scaling the edge density from 0.5% to 90%. The results are shown in Table 5.4 and Figure 5.5.

Density	Edges (m)	A1	A2	A3	A4	A5	A6
0.005	64,831	0.684	1.855	0.308	0.098	0.088	0.023
0.040	494,797	2.87	8.14	0.895	0.417	0.058	0.015
0.150	1,592,650	8.68	26.1	3.30	1.26	0.051	0.019
0.450	4,207,022	24.1	79.4	8.59	4.30	0.042	0.021
0.900	7,132,201	36.0	138	13.0	8.13	0.042	0.014

Table 5.4: The running time (in ms) showing the density impact for a fixed $n = 5000$. The identifiers denote the following algorithms: A1 (MCS Recognition), A2 (LexBFS Recognition), A3 (Vertex Coloring), A4 (Maximum Clique), A5 (Minimum Clique Cover), and A6 (Maximum Independent Set).

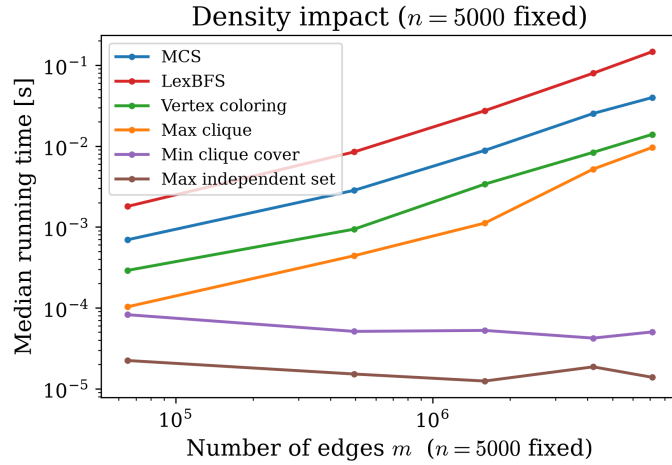


Figure 5.5: Graph of the running time showing density impact for fixed $n = 5000$.

5.4.4 Constant Factor Analysis

An ordinary least-squares linear regression was performed to estimate the constant factor α (in nanoseconds per unit of $n + m$) representing the processing overhead of each algorithm. Across all density tiers, the recognition algorithms exhibit the largest constant overheads, with MCS consistently demonstrating lower overhead than LexBFS. The sparse-tier constants are elevated compared to medium and dense, reflecting cache effects: sparse graphs at large scale have many vertices with small degrees, causing frequent cache misses in the recognition data structures. The optimization algorithms show much smaller constants. Notably, the constant factors for clique cover and independent set drop by multiple orders of magnitude in denser graphs, empirically confirming that these greedy algorithms bypass redundant operations as the search space structurally shrinks.

Algorithm	All Tiers α	Sparse α	Medium α	Dense α
MCS	26.91	69.15	7.087	6.040
LexBFS	59.04	138.84	20.49	20.36
Coloring	11.26	29.45	3.210	1.978
Max Clique	6.245	15.79	2.080	1.313
Min Clique Cover	9.961	30.41	0.1277	0.00756
Max Independent Set	4.042	12.32	0.0676	0.00305

Table 5.5: Estimated constant factor α (in nanoseconds per unit of $n + m$) for the linear regression model $\text{Time} = \alpha(n + m)$ across individual and combined density tiers.

5.5 Conclusions

The running time of the algorithms depends on both the number of edges and the number of vertices. From the figures and tables, it is clear that the algorithms are linear within the tested range, confirming the $O(n + m)$ theoretical complexity.

The analysis of the constant factor α reveals that MCS has lower constant overhead than LexBFS, making it the faster choice for recognition in practice. For the optimization algorithms, while coloring and clique scale directly with the graph size, the clique cover and independent set algorithms benefit from the structural properties of dense graphs, which significantly reduces their execution overhead.

Bibliography

- [1] Gabriel Dirac. “On rigid circuit graphs”. In: *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg* 25 (1961), pp. 71–76.
- [2] Delbert Fulkerson and Oliver Gross. “Incidence matrices and interval graphs”. In: *Pacific Journal of Mathematics* 15.3 (1965), pp. 835–855.
- [3] Andreas Brandstädt, Van Bang Le, and Jeremy Spinrad. *Graph Classes: A Survey*. SIAM, 1999.
- [4] Donald Rose, Robert Tarjan, and George Lueker. “Algorithmic Aspects of Vertex Elimination on Graphs”. In: *SIAM Journal on Computing* 5.2 (1976), pp. 266–283.
- [5] Robert Tarjan and Mihalis Yannakakis. “Simple Linear-Time Algorithms to Test Chordality of Graphs, Test Acyclicity of Hypergraphs, and Selectively Reduce Acyclic Hypergraphs”. In: *SIAM Journal on Computing* 13.3 (1984), pp. 566–579.
- [6] Fănică Gavril. “Algorithms for Minimum Coloring, Maximum Clique, Minimum Covering by Cliques, and Maximum Independent Set of a Chordal Graph”. In: *SIAM Journal on Computing* 1.2 (1972), pp. 180–187.
- [7] Hans Bodlaender. “A tourist guide through treewidth”. In: *Acta Cybernetica* 11 (1993), pp. 1–23.
- [8] Paola Festa, Panos Pardalos, and Mauricio Resende. “Feedback set problems”. In: *Handbook of Combinatorial Optimization*. Ed. by Ding-Zhu Du and Panos M. Pardalos. Vol. Supplement Volume A. Kluwer Academic Publishers, 2000, pp. 209–259.
- [9] V. N. Zemlyachenko, N. M. Korneenko, and R. I. Tyshkevich. “Graph isomorphism problem”. In: *Journal of Soviet Mathematics* 29.4 (1985), pp. 1426–1481.
- [10] Pinar Heggernes, Daniel Lokshtanov, Rodica Mihal, and Charis Papadopoulos. “Cutwidth of Split Graphs, Threshold Graphs, and Proper Interval Graphs”. In: *Graph-Theoretic Concepts in Computer Science (WG 2008)*. Vol. 5344. Lecture Notes in Computer Science. Springer, 2008, pp. 218–229.
- [11] Kellogg Booth and J. Howard Johnson. “Dominating sets in chordal graphs”. In: *SIAM Journal on Computing* 11.1 (1982), pp. 191–199.

- [12] Charles Colbourn and Lorna Stewart. “Dominating cycles in series-parallel graphs”. In: *Ars Combinatoria* 19A (1985), pp. 107–112.
- [13] Hans Bodlaender and Klaus Jansen. “On the complexity of the maximum cut problem”. In: *Nordic Journal of Computing* 7.1 (2000), pp. 14–31.
- [14] Béla Bollobás. *Modern Graph Theory*. Graduate Texts in Mathematics 184. Springer-Verlag New York, 1998.
- [15] Claude Berge. “Färbung von Graphen, deren sämtliche bzw. deren ungerade Kreise starr sind”. In: *Wissenschaftliche Zeitschrift der Martin-Luther-Universität Halle-Wittenberg, Mathematisch-Naturwissenschaftliche Reihe* 10 (1961), pp. 114–115.
- [16] Tinaz Ekim, Mordechai Shalom, and Oylum Şeker. “The complexity of subtree intersection representation of chordal graphs and linear time chordal graph generation”. In: *Journal of Combinatorial Optimization* 41 (2021), pp. 710–735.
- [17] OEIS Foundation Inc. *The On-Line Encyclopedia of Integer Sequences, Sequence A048192*. <https://oeis.org/A048192>. 2026.